

# Artist Music Dashboard

*Milena Martínez*

*Jul 2026*

# 1- Interacción significativa con la IA

## [Mock Data]

**Objetivo** Diseñar y proveer una arquitectura de datos ficticia ([mockData.ts](https://mockdata.ts)), relacional y temporal (2021–2026), que sirva como fuente para el *Music Artist Dashboard* interactivo

**Herramienta de IA utilizada:** Chat GPT

### Prompt/Approach:

#### - Que le preguntaste?

Actúa como un **\*\*Senior Frontend Engineer\*\*** realizando la prueba técnica (\*Take-Home Assignment\* Asignación).

Actualmente estás diseñando y generando el archivo de Mock Data- Este archivo alimentará todos los componentes visuales, KPIs, tablas y gráficos interactivos del dashboard.

---

#### Misión:

Genera un conjunto de datos ficticios pero coherentes, estructurados en tres colecciones principales: `artists`, `albums` y `songs`, cubriendo información desde 2021 hasta 2026.

---

#### Necesito\_

1. Modelado Relacional (PK / FK): Usar IDs únicos numéricos, con logica
2. Coherencia Matemática:
  - $\text{totalLikes} = \text{spotifyLikes} + \text{youtubeLikes} + \text{appleMusicLikes}$
  - $\text{totalViews} = \text{spotifyViews} + \text{youtubeViews} + \text{appleMusicViews}$
  - $\text{ROI} = ((\text{Revenue} - \text{ProductionCost}) / \text{ProductionCost}) * 100$  (calculado con precisión).
3. Artistas requeridos: Sabrina Carpenter, Ariana Grande, Bruno Mars, Bad Bunny, Tyler the Creator, Olivia Rodrigo, Dua Lipa, BTS, Lady Gaga.
4. NEcesario (Sabrina Carpenter):
  - Debe ser la primera artista del dataset.
  - El bannerAlbum configurado en su perfil debe ser "Espresso", con sus estadísticas y portadas correspondientes.
5. Realismo: Nombres de artistas, canciones y álbumes reales de sus discografías (2021-2026).  
Números de reproducciones, ventas e ingresos inventados pero con proporciones realistas.

---

#### Estructura:

```
export interface SongData {  
  id: string;  
  artistId: string;  
  albumId: string;  
  titlesong: string;
```

```

    coverurl: string;
    releaseYear: number;
    spotifyLikes: number;
    youtubeLikes: number;
    appleMusicLikes: number;
    spotifyViews: number;
    youtubeViews: number;
    appleMusicViews: number;
    totalLikes: number;
    totalViews: number;
    kpis: {
        earnings: number;
        microMonetization: number;
        conversionRate: number; // Porcentaje 0-100
    };
}

```

```

export interface AlbumData {
    id: string;
    artistId: string;
    titlealbum: string;
    coverurl: string;
    description: string;
    releaseYear: number;
    revenue: number;
    productionCost: number;
    kpis: {
        breakEvenUnits: number;
        roi: number; // Calculado: ((revenue - productionCost) / productionCost) * 100
    };
    graphics: {
        offlineProfits: number;
        onlineProfits: number;
    };
}

```

```

export interface ArtistData {
    id: string;
    name: string;
    avatarUrl: string;
    artistPhotoUrl: string;
    bannerAlbum: {
        albumId: string;
        titlealbum: string;
        coverurl: string;
        totalLikes: number;
        description: string;
    };
    kpis: {
        conversionRate: number; // % oyentes que compran merch/tickets
    };
}

```

```

    subscriptionEarning: number;
    netBenefits: number; // Beneficio neto del último lanzamiento
};
profitableTracks: string[]; // Top 5 títulos de canciones más rentables
globalReach: { country: string; listeners: number }[]; // Top 10 países
graphics: {
    albumEarning: number;
    merchEarning: number;
    showsEarning: number;
    streamingEarning: number;
    youtubeFollows: number;
    spotifyFollows: number;
    appleMusicFollows: number;
};
}

export interface MockDataSet {
    artists: ArtistData[];
    albums: AlbumData[];
    songs: SongData[];
}

```

### - How did you frame the problem?

El problema se abordó identificando que las necesidades del dashboard (Releases, Sales, Fan Engagement) no son métricas aisladas, sino un ecosistema interconectado, por lo que se diseñó un esquema relacional normalizado mediante (PK/FK) separando Artistas, Álbumes y Canciones

### **\*\*Resultados:\*\***

#### **-Que te dio la IA?**

La estructura que planteas tiene buena base, pero hay un problema de modelado: estás mezclando información de distintas entidades (por ejemplo **BannerAlbum** dentro del artista o KPIs calculados dentro de cada registro). Si este fuera un proyecto real de frontend, intentaría que la mock data simulara una base de datos relacional para que luego sea mucho más sencillo hacer filtros, gráficas y joins.

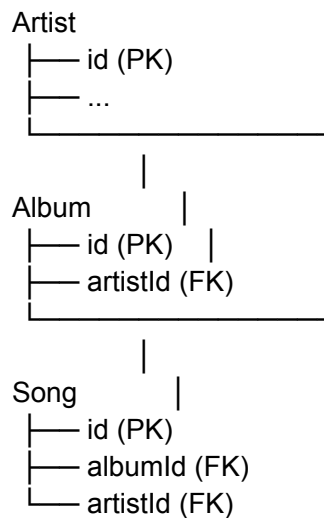
Yo lo estructuraría así:

- **ArtistData** → información del artista
- **AlbumData** → información de álbumes (FK → Artist)
- **SongData** → información de canciones (FK → Album y Artist)

Y dejaría los KPIs calculados directamente en la mock data porque el ejercicio no requiere un backend.

---

## Relaciones



De esta forma puedes hacer cosas como

```
const artistAlbums = albums.filter(a => a.artistId === artist.id);
```

```
const artistSongs = songs.filter(s => s.artistId === artist.id);
```

```
const albumSongs = songs.filter(s => s.albumId === album.id);
```

Mucho más limpio que relacionar por nombre.

---

## Artistas

Te recomiendo estos artistas.

| id | Artista            |
|----|--------------------|
| 1  | Sabrina Carpenter  |
| 2  | Ariana Grande      |
| 3  | Bruno Mars         |
| 4  | Bad Bunny          |
| 5  | Tyler, The Creator |
| 6  | Olivia Rodrigo     |
| 7  | Dua Lipa           |
| 8  | BTS                |

## Álbum elegido por artista

Para que haya registros entre 2021-2026.

| Año  | Artista            | Álbum                                                                   |
|------|--------------------|-------------------------------------------------------------------------|
| 2024 | Sabrina Carpenter  | Short n' Sweet ( <i>Banner Album: Espresso como canción destacada</i> ) |
| 2024 | Ariana Grande      | Eternal Sunshine                                                        |
| 2021 | Bruno Mars         | An Evening with Silk Sonic                                              |
| 2023 | Bad Bunny          | nadie sabe lo que va a pasar mañana                                     |
| 2025 | Tyler, The Creator | Chromakopia                                                             |
| 2023 | Olivia Rodrigo     | GUTS                                                                    |
| 2024 | Dua Lipa           | Radical Optimism                                                        |
| 2022 | BTS                | Proof                                                                   |
| 2025 | Lady Gaga          | Mayhem                                                                  |

---

## Canciones (5 por artista)

### Sabrina Carpenter

- Espresso
  - Please Please Please
  - Taste
  - Feather
  - Nonsense
- 

### Ariana Grande

- we can't be friends
- yes, and?

- the boy is mine
  - supernatural
  - imperfect for you
- 

## **Bruno Mars**

- Leave The Door Open
  - Smokin Out The Window
  - Skate
  - Fly As Me
  - After Last Night
- 

## **Bad Bunny**

- Monaco
  - Fina
  - Perro Negro
  - Hibiki
  - Where She Goes
- 

## **Tyler, The Creator**

- Sticky
  - Noid
  - Darling, I
  - Like Him
  - Judge Judy
- 

## **Olivia Rodrigo**

- vampire
  - bad idea right?
  - get him back!
  - obsessed
  - all-american bitch
- 

## **Dua Lipa**

- Houdini
  - Training Season
  - Illusion
  - These Walls
  - Falling Forever
- 

## BTS

- Yet To Come
  - Run BTS
  - For Youth
  - Butter
  - Permission To Dance
- 

## Lady Gaga

- Disease
  - Abracadabra
  - Garden of Eden
  - Perfect Celebrity
  - Vanish Into You
- 

## Años

Te recomiendo que todas las tablas tengan una propiedad

year: 2021 | 2022 | 2023 | 2024 | 2025 | 2026

Así podrás filtrar el dashboard fácilmente.

---

## Valores lógicos

### Streaming

Los ingresos podrían moverse en estos rangos



| Tipo      | Rango      |
|-----------|------------|
| Streaming | 4M - 40M   |
| Merch     | 1M - 20M   |
| Tours     | 10M - 180M |
| Albums    | 2M - 30M   |

---

## Conversion Rate

Muy importante.

No pongas cosas como 35%.

Una conversión realista es

1.2%  
2.8%  
3.4%  
4.7%  
5.9%

Máximo alrededor de

7%

---

## ROI

Entre

120%

240%

380%

520%

Nunca negativos porque todos son artistas exitosos.

---

## Break Even

Ejemplo

Production Cost

8,000,000

Revenue

31,000,000

Precio promedio álbum

15 USD

Break Even

533,334 unidades

---

## Totallikes

Siempre calcula

totalLikes =  
spotifyLikes +  
youtubeLikes +  
appleMusicLikes

Nunca inventes el resultado.

---

## Total Views

totalViews =  
spotifyViews +  
youtubeViews +  
appleMusicViews

---

## Banner Album de Sabrina

Yo lo dejaría así

```
bannerAlbum: {  
  albumId: 1,  
  titleAlbum: "Short n' Sweet",  
  featuredTrack: "Espresso",
```

```
coverUrl: "...",
totalLikes: 128540000,
description:
  "Sabrina Carpenter's sixth studio album featuring the worldwide hit 'Espresso', blending pop,
  disco and country influences."
}
```

Es más coherente porque **Espresso no es un álbum**, sino una canción del álbum *Short n' Sweet*. Así mantienes la lógica del modelo mientras destacas el tema principal en el banner.

---

## Mi recomendación

Para un puesto **Senior Frontend**, intentaría que la mock data parezca una API real. En lugar de un único archivo enorme con objetos anidados, crearía tres archivos independientes:

mock/

artists.ts  
albums.ts  
songs.ts

Cada entidad usaría IDs para las relaciones (**artistId**, **albumId**) y todos los valores derivados (**totalLikes**, **totalViews**, **ROI**, **NetBenefits**, etc.) serían matemáticamente consistentes con los datos base. Esto facilitará construir componentes reutilizables, filtros por año (2021-2026), gráficas y widgets sin tener que transformar excesivamente los datos, y dará una impresión mucho más cercana a un entorno de producción.

### ¿Lo usaste tal cual o lo modificaste?

Luego de pedirle que cambiara la estructura del banner de albums, a que fuera un banner de canciones, me dio el resultado en 3 archivos descargables y esos si los use tal cual

### ¿Por qué tomaste esas decisiones?

La IA dio un par de recomendaciones que seguí al pie de la letra:

- Estructurar la mock data y relacionarla con ID de la manera que recomendó fue menester para evitar duplicación de datos y acoplamiento innecesario
- Para una mejor legibilidad y evitar archivos pesados, hacer una arquitectura modular separando la mock data en tres archivos ("Artists.ts", "songs.ts", "albums.ts") me pareció importante
- Y la modificación del banner de Espresso, ya que esta es una canción y no un álbum como pensaba, lo que hacía un conflicto en la estructura.

### **\*\*Aprendizaje:\*\***

#### ¿Qué aprendiste de esta interacción?

Aprendí que el primer paso para construir una interfaz fluida no es escribir el código ni diseñar los componentes, sino diseñar un modelo de datos coherente y acorde para el tipo de trabajo que se está realizando.

¿Haría algo de manera diferente?

---

## 2- Interacción significativa con la IA

### [Página de bienvenida, menú y dashboard general]

**Objetivo** Desarrollar la vista inicial de bienvenida y el dashboard principal (*Main Dashboard*) en Next.js 16+ con Tailwind CSS, con un buscador centralizado, secciones de ranking (Top 3 Álbumes "Cloud" y Top 5 Artistas en contenedor *Glassmorphism*) y exactamente dos gráficos analíticos de negocio.

**Herramienta de IA utilizada:** Gemini

#### **Prompt/Approach:**

##### **- Que le preguntaste?**

Actúa como un Senior Frontend Engineer altamente experimentado en el ecosistema Next.js , React, TypeScript y Tailwind CSS. No requieres asistencia de la IA y sigues especificaciones de diseño al pie de la letra.

Estás construyendo la interfaz para el **\*\*Take-Home Assignment\*\*** [Adjunta Asignacion]

---

#### **# Fuentes de Datos Disponibles**

Ya contamos con los datos simulados normalizados en tres archivos separados:

1. artists.ts (Colección de artistas, métricas globales y banner track)

```
export interface Artist {  
  id:number;  
  year:number;  
  name:string;  
  avatarUrl:string;  
  artistPhoto:string;  
  bannerSong:{  
    songId:number;  
    titleSong:string;  
    albumId:number;  
    album:string;  
    coverUrl:string;  
    totalLikes:number;  
    description:string;  
  };  
  kpis:{  
    conversionRate:number;  
    subscriptionEarning:number;  
    netBenefits:number;  
  };  
  profitableTracks:string[];  
  globalReach:string[];
```

```

graphics:{
  albumEarning:number;
  merchEarning:number;
  showsEarning:number;
  streamingEarning:number;
  youtubeFollows:number;
  spotifyFollows:number;
  appleMusicFollows:number;
};
}

```

```

export const artists: Artist[] = [
{
  id:1,year:2024,name:"Sabrina Carpenter",
  avatarUrl:"https://picsum.photos/seed/sabrina-avatar/200",
  artistPhoto:"https://picsum.photos/seed/sabrina-photo/1200/800",
  bannerSong:{songId:1,titleSong:"Espresso",albumId:1,album:"Short n'
Sweet",coverUrl:"https://picsum.photos/seed/espresso/600",totalLikes:128000000,description:"Global hit single from Short n' Sweet."},
  kpis:{conversionRate:4.9,subscriptionEarning:1800000,netBenefits:26500000},
  profitableTracks:["Espresso","Please Please Please","Taste","Feather","Nonsense"],
  globalReach:["USA","UK","Canada","Australia","Germany","Brazil","Mexico","France","Japan","Spain"],
  graphics:{albumEarning:31000000,merchEarning:8200000,showsEarning:47000000,streamingEarning:22500000,youtubeFollows:2100000,spotifyFollows:3600000,appleMusicFollows:1200000}
},
{
  id:2,year:2024,name:"Ariana Grande",
  avatarUrl:"https://picsum.photos/seed/ariana-avatar/200",
  artistPhoto:"https://picsum.photos/seed/ariana-photo/1200/800",
  bannerSong:{songId:2,titleSong:"we can't be friends",albumId:2,album:"Eternal Sunshine",coverUrl:"https://picsum.photos/seed/ariana/600",totalLikes:119000000,description:"Lead single from Eternal Sunshine."},
  kpis:{conversionRate:5.2,subscriptionEarning:2400000,netBenefits:34200000},
  profitableTracks:["we can't be friends","yes, and?","7 rings","positions","Into You"],
  globalReach:["USA","UK","Brazil","Canada","Mexico","Japan","France","Germany","Italy","Australia"],
  graphics:{albumEarning:36000000,merchEarning:9100000,showsEarning:58000000,streamingEarning:28000000,youtubeFollows:1900000,spotifyFollows:3200000,appleMusicFollows:1300000}
},
{
  id:3,year:2021,name:"Bruno Mars",
  avatarUrl:"https://picsum.photos/seed/bruno-avatar/200",
  artistPhoto:"https://picsum.photos/seed/bruno-photo/1200/800",
  bannerSong:{songId:3,titleSong:"Leave The Door Open",albumId:3,album:"An Evening with Silk Sonic",coverUrl:"https://picsum.photos/seed/bruno/600",totalLikes:110000000,description:"Award-winning Silk Sonic single."},
  kpis:{conversionRate:5.6,subscriptionEarning:1700000,netBenefits:30100000},
  profitableTracks:["Leave The Door Open","Smokin Out The Window","24K Magic","That's What I Like","Locked Out of Heaven"],

```

```

globalReach:["USA","Japan","Brazil","UK","Philippines","Canada","Australia","France","Germany",
,"Mexico"],
graphics:{albumEarning:28000000,merchEarning:7200000,showsEarning:61000000,streamingE
arning:26000000,youtubeFollows:1600000,spotifyFollows:2800000,appleMusicFollows:950000}
},
{
id:4,year:2023,name:"Bad Bunny",
avatarUrl:"https://picsum.photos/seed/badbunny-avatar/200",
artistPhoto:"https://picsum.photos/seed/badbunny-photo/1200/800",
bannerSong:{songId:4,titleSong:"Monaco",albumId:4,album:"nadie sabe lo que va a pasar
mañana",coverUrl:"https://picsum.photos/seed/monaco/600",totalLikes:98000000,description:"Hit
track from the 2023 album."},
kpis:{conversionRate:5.8,subscriptionEarning:2900000,netBenefits:39000000},
profitableTracks:["Monaco","Moscow Mule","Tití Me Preguntó","WHERE SHE GOES","Dakiti"],
globalReach:["Mexico","USA","Spain","Colombia","Argentina","Chile","Peru","Brazil","Canada","It
aly"],
graphics:{albumEarning:42000000,merchEarning:11000000,showsEarning:99000000,streaming
Earning:35000000,youtubeFollows:2400000,spotifyFollows:4200000,appleMusicFollows:160000
0}
},
{
id:5,year:2025,name:"Tyler, The Creator",
avatarUrl:"https://picsum.photos/seed/tyler-avatar/200",
artistPhoto:"https://picsum.photos/seed/tyler-photo/1200/800",
bannerSong:{songId:5,titleSong:"Sticky",albumId:5,album:"Chromakopia",coverUrl:"https://picsu
m.photos/seed/sticky/600",totalLikes:42000000,description:"Featured single."},
kpis:{conversionRate:3.8,subscriptionEarning:900000,netBenefits:18400000},
profitableTracks:["Sticky","EARFQUAKE","See You Again","Noid","NEW MAGIC WAND"],
globalReach:["USA","Canada","UK","Australia","Germany","France","Japan","Mexico","Netherlan
ds","Brazil"],
graphics:{albumEarning:22000000,merchEarning:5300000,showsEarning:33000000,streamingE
arning:17000000,youtubeFollows:800000,spotifyFollows:1700000,appleMusicFollows:500000}
},
{
id:6,year:2023,name:"Olivia
Rodrigo",avatarUrl:"https://picsum.photos/seed/olivia-avatar/200",artistPhoto:"https://picsum.phot
os/seed/olivia-photo/1200/800",
bannerSong:{songId:6,titleSong:"vampire",albumId:6,album:"GUTS",coverUrl:"https://picsum.pho
tos/seed/vampire/600",totalLikes:93000000,description:"Lead single from GUTS."},
kpis:{conversionRate:4.6,subscriptionEarning:1400000,netBenefits:23000000},
profitableTracks:["vampire","drivers license","good 4 u","traitor","bad idea right?"],
globalReach:["USA","UK","Canada","Australia","Philippines","Japan","Germany","France","Brazil",
,"Mexico"],
graphics:{albumEarning:26000000,merchEarning:6000000,showsEarning:39000000,streamingE
arning:21000000,youtubeFollows:1200000,spotifyFollows:2300000,appleMusicFollows:850000}
},
{
id:7,year:2024,name:"Dua
Lipa",avatarUrl:"https://picsum.photos/seed/dua-avatar/200",artistPhoto:"https://picsum.photos/se
ed/dua-photo/1200/800",

```

```

    bannerSong:{songId:7,titleSong:"Houdini",albumId:7,album:"Radical
    Optimism",coverUrl:"https://picsum.photos/seed/houdini/600",totalLikes:85000000,description:"D
    ance-pop hit."},
    kpis:{conversionRate:4.7,subscriptionEarning:1600000,netBenefits:25100000},
    profitableTracks:["Houdini","Levitating","Dance The Night","Training Season","Don't Start Now"],
    globalReach:["UK","USA","Germany","France","Italy","Spain","Brazil","Australia","Canada","Japa
    n"],
    graphics:{albumEarning:29000000,merchEarning:6800000,showsEarning:48000000,streamingE
    arning:24000000,youtubeFollows:1500000,spotifyFollows:2900000,appleMusicFollows:950000}
  },
  {
    id:8,year:2022,name:"BTS",avatarUrl:"https://picsum.photos/seed/bts-avatar/200",artistPhoto:"htt
    ps://picsum.photos/seed/bts-photo/1200/800",
    bannerSong:{songId:8,titleSong:"Yet To
    Come",albumId:8,album:"Proof",coverUrl:"https://picsum.photos/seed/proof/600",totalLikes:1020
    00000,description:"Anthology title track."},
    kpis:{conversionRate:6.1,subscriptionEarning:3500000,netBenefits:45000000},
    profitableTracks:["Yet To Come","Butter","Dynamite","Run BTS","Permission to Dance"],
    globalReach:["South
    Korea","Japan","USA","Philippines","Indonesia","Brazil","Mexico","India","UK","Thailand"],
    graphics:{albumEarning:47000000,merchEarning:14000000,showsEarning:120000000,streamin
    gEarning:38000000,youtubeFollows:3100000,spotifyFollows:5100000,appleMusicFollows:19000
    00}
  },
  {
    id:9,year:2025,name:"Lady
    Gaga",avatarUrl:"https://picsum.photos/seed/gaga-avatar/200",artistPhoto:"https://picsum.photos
    /seed/gaga-photo/1200/800",
    bannerSong:{songId:9,titleSong:"Disease",albumId:9,album:"MAYHEM",coverUrl:"https://picsum.
    photos/seed/gaga/600",totalLikes:74000000,description:"Lead single from MAYHEM."},
    kpis:{conversionRate:5.1,subscriptionEarning:2100000,netBenefits:32000000},
    profitableTracks:["Disease","Shallow","Poker Face","Bad Romance","Rain On Me"],
    globalReach:["USA","UK","Brazil","Germany","France","Italy","Canada","Australia","Japan","Spai
    n"],
    graphics:{albumEarning:34000000,merchEarning:8300000,showsEarning:70000000,streamingE
    arning:27000000,youtubeFollows:1800000,spotifyFollows:3100000,appleMusicFollows:1100000}
  }
];

```

## 2. `albums.ts` (Álbumes, ROI, ganancias online/offline, break-even)

```

export interface Album {
  id:number;
  artistId:number;
  year:number;
  titleAlbum:string;
  coverUrl:string;
  description:string;
  revenue:number;
  productionCost:number;
  breakEven:number;
}

```

```
    roi:number;
    graphics:{
      offlineProfits:number;
      onlineProfits:number;
    };
  }
}
```

```
export const albums: Album[] = [
  {id:1,artistId:1,year:2024,titleAlbum:"Short n'
Sweet",coverUrl:"https://picsum.photos/seed/album1/600",description:"Sixth studio album by
Sabrina
Carpenter.",revenue:38000000,productionCost:11500000,breakEven:766667,roi:230.43,graphics
:{offlineProfits:9000000,onlineProfits:29000000}},
  {id:2,artistId:2,year:2024,titleAlbum:"Eternal
Sunshine",coverUrl:"https://picsum.photos/seed/album2/600",description:"Seventh studio album
by Ariana
Grande.",revenue:46000000,productionCost:12000000,breakEven:800000,roi:283.33,graphics:{o
fflineProfits:12000000,onlineProfits:34000000}},
  {id:3,artistId:3,year:2021,titleAlbum:"An Evening with Silk
Sonic",coverUrl:"https://picsum.photos/seed/album3/600",description:"Collaboration album by
Silk
Sonic.",revenue:35000000,productionCost:10000000,breakEven:666667,roi:250.00,graphics:{offl
ineProfits:11000000,onlineProfits:24000000}},
  {id:4,artistId:4,year:2023,titleAlbum:"nadie sabe lo que va a pasar
mañana",coverUrl:"https://picsum.photos/seed/album4/600",description:"Bad Bunny's
chart-topping
release.",revenue:54000000,productionCost:14000000,breakEven:933334,roi:285.71,graphics:{o
fflineProfits:10000000,onlineProfits:44000000}},

  {id:5,artistId:5,year:2025,titleAlbum:"Chromakopia",coverUrl:"https://picsum.photos/seed/album5/
600",description:"Tyler, The Creator studio
album.",revenue:29000000,productionCost:9000000,breakEven:600000,roi:222.22,graphics:{offli
neProfits:7000000,onlineProfits:22000000}},

  {id:6,artistId:6,year:2023,titleAlbum:"GUTS",coverUrl:"https://picsum.photos/seed/album6/600",d
escription:"Olivia Rodrigo second studio
album.",revenue:34000000,productionCost:10000000,breakEven:666667,roi:240.00,graphics:{off
lineProfits:9000000,onlineProfits:25000000}},
  {id:7,artistId:7,year:2024,titleAlbum:"Radical
Optimism",coverUrl:"https://picsum.photos/seed/album7/600",description:"Third studio album by
Dua
Lipa.",revenue:36000000,productionCost:11000000,breakEven:733334,roi:227.27,graphics:{offlin
eProfits:9000000,onlineProfits:27000000}},

  {id:8,artistId:8,year:2022,titleAlbum:"Proof",coverUrl:"https://picsum.photos/seed/album8/600",de
scription:"BTS anthology
album.",revenue:60000000,productionCost:15000000,breakEven:1000000,roi:300.00,graphics:{o
fflineProfits:18000000,onlineProfits:42000000}},

  {id:9,artistId:9,year:2025,titleAlbum:"MAYHEM",coverUrl:"https://picsum.photos/seed/album9/600
```



```

",description:"Lady Gaga studio
album.",revenue:43000000,productionCost:12000000,breakEven:800000,roi:258.33,graphics:{off
lineProfits:12000000,onlineProfits:31000000}}
];

```

### 3. `songs.ts` (Canciones, métricas por plataforma Spotify/YouTube/Apple Music)

```

export interface Song {
  id:number;
  artistId:number;
  albumId:number;
  year:number;
  titleSong:string;
  coverUrl:string;
  spotifyLikes:number;
  youtubeLikes:number;
  appleMusicLikes:number;
  spotifyViews:number;
  youtubeViews:number;
  appleMusicViews:number;
  totalLikes:number;
  totalViews:number;
  kpis:{
    earnings:number;
    microMonetization:number;
    conversionRate:number;
  };
}

```

```

export const songs: Song[] = [

```

```

  {id:1,artistId:1,albumId:1,year:2024,titleSong:"Espresso",coverUrl:"https://picsum.photos/seed/song1/600",spotifyLikes:52000000,youtubeLikes:43000000,appleMusicLikes:33000000,spotifyViews:980000000,youtubeViews:710000000,appleMusicViews:420000000,totalLikes:128000000,totalViews:2110000000,kpis:{earnings:22500000,microMonetization:3200000,conversionRate:4.9}},

```

```

  {id:2,artistId:2,albumId:2,year:2024,titleSong:"we can't be friends",coverUrl:"https://picsum.photos/seed/song2/600",spotifyLikes:50000000,youtubeLikes:39000000,appleMusicLikes:30000000,spotifyViews:920000000,youtubeViews:650000000,appleMusicViews:390000000,totalLikes:119000000,totalViews:1960000000,kpis:{earnings:20800000,microMonetization:2800000,conversionRate:5.2}},

```

```

  {id:3,artistId:3,albumId:3,year:2021,titleSong:"Leave The Door Open",coverUrl:"https://picsum.photos/seed/song3/600",spotifyLikes:46000000,youtubeLikes:36000000,appleMusicLikes:28000000,spotifyViews:880000000,youtubeViews:640000000,appleMusicViews:360000000,totalLikes:110000000,totalViews:1880000000,kpis:{earnings:19400000,microMonetization:2400000,conversionRate:5.6}},

```

```

  {id:4,artistId:4,albumId:4,year:2023,titleSong:"Monaco",coverUrl:"https://picsum.photos/seed/song4/600",spotifyLikes:42000000,youtubeLikes:31000000,appleMusicLikes:25000000,spotifyViews:790000000,youtubeViews:540000000,appleMusicViews:330000000,totalLikes:98000000,totalViews:1660000000,kpis:{earnings:18700000,microMonetization:2600000,conversionRate:5.8}},

```

```
{id:5,artistId:5,albumId:5,year:2025,titleSong:"Sticky",coverUrl:"https://picsum.photos/seed/song5/600",spotifyLikes:18000000,youtubeLikes:14000000,appleMusicLikes:10000000,spotifyViews:350000000,youtubeViews:210000000,appleMusicViews:150000000,totalLikes:42000000,totalViews:710000000,kpis:{earnings:9800000,microMonetization:1100000,conversionRate:3.8}},
```

```
{id:6,artistId:6,albumId:6,year:2023,titleSong:"vampire",coverUrl:"https://picsum.photos/seed/song6/600",spotifyLikes:39000000,youtubeLikes:30000000,appleMusicLikes:24000000,spotifyViews:760000000,youtubeViews:520000000,appleMusicViews:310000000,totalLikes:93000000,totalViews:1590000000,kpis:{earnings:17600000,microMonetization:2200000,conversionRate:4.6}},
```

```
{id:7,artistId:7,albumId:7,year:2024,titleSong:"Houdini",coverUrl:"https://picsum.photos/seed/song7/600",spotifyLikes:35000000,youtubeLikes:28000000,appleMusicLikes:22000000,spotifyViews:680000000,youtubeViews:470000000,appleMusicViews:280000000,totalLikes:85000000,totalViews:1430000000,kpis:{earnings:16500000,microMonetization:2100000,conversionRate:4.7}},
```

```
{id:8,artistId:8,albumId:8,year:2022,titleSong:"Yet To Come",coverUrl:"https://picsum.photos/seed/song8/600",spotifyLikes:43000000,youtubeLikes:34000000,appleMusicLikes:25000000,spotifyViews:820000000,youtubeViews:600000000,appleMusicViews:350000000,totalLikes:102000000,totalViews:1770000000,kpis:{earnings:20100000,microMonetization:2900000,conversionRate:6.1}},
```

```
{id:9,artistId:9,albumId:9,year:2025,titleSong:"Disease",coverUrl:"https://picsum.photos/seed/song9/600",spotifyLikes:30000000,youtubeLikes:25000000,appleMusicLikes:19000000,spotifyViews:610000000,youtubeViews:430000000,appleMusicViews:240000000,totalLikes:74000000,totalViews:1280000000,kpis:{earnings:14900000,microMonetization:1800000,conversionRate:5.1}}];
```

---

## # Misión Actual: Implementación del Landing & Main Dashboard Layout

Crea la estructura de componentes en Next.js 16+ para la Pantalla 1: (Bienvenida/Landing) y la Pantalla 2 (Dashboard General) respetando de forma estricta las siguientes reglas de layout y diseño:

### 1. Bienvenida

- Visuales: Fondo con degradado (gradient effect) estilizado.
- Branding/Logo: Tipografía principal Inter en color blanco.
- Logotipo de la Marca: tres discos de vinilo interconectados: uno en el centro sobresaliendo en primer plano, parcialmente sobre los dos discos traseros a ambos lados.

### 2. Main Dashboard View (Scroll Down)

Al hacer scroll hacia abajo se revela el dashboard global con la siguiente descripción:

Central:

- Barra de búsqueda interactiva (\*Searchbar\*) para filtrar por nombre de Artista, Canción o Álbum.

Top 3 Álbumes (Destacados):

- Muestra un Top 3 de álbumes con avatares circulares flanqueados por la tipografía decorativa "cl" a la izquierda y "d" a la derecha, formando la palabra icónica "cloud".

Top 5 Artistas (Glassmorphism Container):

- Tarjeta con efecto de cristal translúcido (glassmorphism) que muestra los avatares circulares del Top 5 de Artistas más rankeados.

Sección de Analytics (2 Gráficos ÚNICAMENTE):

- Gráfico Izquierdo (Active vs. Deep Catalog): Gráfico de pastel/donas o barras acumuladas que calcula el % de ingresos totales entre lanzamientos recientes (Frontline\*, \$<18\$ meses) vs. catálogo histórico (\*Deep Catalog\*, \$>18\$ meses) usando la data de songs y albums.

- Gráfico Derecho (Físico vs. Streaming por Genero): Comparativa del ingreso promedio por ventas físicas (offlineProfits) frente a ingresos digitales (onlineProfits).

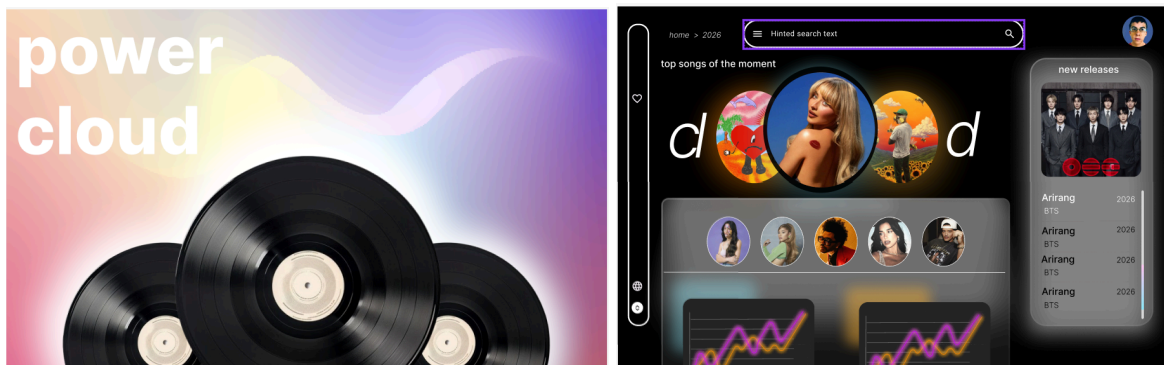
Barra Lateral Izquierda (Sidebar): Menú de navegación principal.

Barra Lateral Derecha: Lista con el feed de "Últimos Lanzamientos" (\*Recent Releases\*).

---

## Reglas

1. Tipografía: Aplicar la fuente Inter en sus diferentes pesos (font-sans, font-bold, font-medium).
2. Alcance: Construye únicamente lo descrito en estas dos pantallas. No agregues gráficos adicionales, páginas secundarias ni componentes que no hayan sido solicitados.
3. Código Limpio: Organiza los componentes en estructuras reutilizables



## - How did you frame the problem?

El problema se planteó desde la fidelidad del diseño y la experiencia del usuario fluida: Se estructuró un borrador visual mostrando la esencia de la marca desde el primer momento.

**\*\*Resultados:\*\***

-Que te dio la IA?

¡He terminado la primera versión de tu página de bienvenida y del panel de control de artistas musicales **Power Cloud**! La aplicación ha sido construida con una atención extrema a los detalles visuales de tus bocetos, con un rendimiento impecable en React 19, TypeScript, Tailwind CSS v4, y animaciones avanzadas de `motion`.

Aquí tienes un resumen conceptual y funcional de todo lo implementado:

## 🎨 Concepto de Diseño y Flujo Visual

## 1. Pantalla de Bienvenida (Primer Fold / Landing):

- **Gradiente y Ondas de Fondo:** Implementé un gradiente dinámico pastel (rosa melocotón a lavanda violeta y azul celeste) de alta gama con un fondo de ondas vectoriales SVG estilizadas en modo overlay.
- **Tipografía "power cloud":** Se presenta en letras blancas gigantes y elegantes con un sutil efecto de brillo (`text-glow`).
- **La Silueta del Logotipo (Trío de Vinilos):** Diseñé tres discos de vinilo realistas en HTML/CSS utilizando gradientes cónicos para simular el brillo metálico de los surcos de vinilo tradicionales. El vinilo del centro es más grande y se superpone parcialmente sobre los dos de los costados, recreando fielmente tu boceto. Además, se incluye un botón interactivo para poner a **girar los vinilos en tiempo real** con velocidades independientes.
- **Indicador de Desplazamiento:** Un botón sutil con animación de rebote que permite al usuario bajar de forma fluida mediante scroll suave (`scrollIntoView`) hacia el panel principal.

2.

## 3. Dashboard Principal (Pantalla Principal "Power Cloud"):

- **Fondo Oscuro y Brillo de Neón:** Un lienzo oscuro profundo (`bg-[#060606]`) complementado con dos grandes auras de luz de neón rosa y naranja detrás del contenido para dar un aire premium y de alta fidelidad.
- **Menú Izquierdo Estilizado:** Una cápsula vertical translúcida muy fina de efecto cristal con los iconos exactos de tu boceto (corazón para favoritos, globo para alcance global y flechas para ordenación), con efectos hover y tooltips elegantes.
- **Buscador Central Reactivo:** Diseñado con efectos de cristal templado. No solo es estético, sino que es completamente **funcional**: al escribir el nombre de un artista, canción o álbum, se despliega un panel de resultados interactivo que permite al usuario seleccionar cualquier coincidencia y actualizar todo el panel al instante.
- **El Elemento Visual "cloud":** Recreado de forma inteligente y sumamente creativa: el texto "c\_l" a la izquierda, seguido de tres avatares circulares flotantes y brillantes de los álbumes mejor posicionados del momento (Sabrina Carpenter, Bad Bunny, Tyler, The Creator), y la letra "d" a la derecha, para formar la palabra **cloud** de forma visual. La tarjeta central de Sabrina Carpenter es más grande y cuenta con un resplandor dorado intenso.
- **Top 5 Artistas en Efecto Glass:** Un contenedor de cristal translúcido que muestra los 5 artistas más rankeados ordenados por prioridad (Olivia Rodrigo, Ariana Grande, Bad Bunny, Dua Lipa, Bruno Mars). Al hacer clic en cualquiera de ellos, **todo el panel se actualiza dinámicamente**: cambia el artista

seleccionado, sus KPIs financieros en vivo, y los gráficos de analíticas correspondientes.

- **Gráfico Izquierdo (Active vs Deep Catalog):** Una visualización de área brillante (`AreaChart` de Recharts) con ondas de color rosa y naranja de neón. El gráfico calcula dinámicamente, a partir de tu base de datos de canciones, la proporción de ganancias entre lanzamientos Frontline (recientes, de 18 meses o menos) y catálogo histórico (Deep Catalog).
- **Gráfico Derecho (Physical vs Streaming):** Un gráfico de barras duales (`BarChart` de Recharts) con degradados de neón naranja y púrpura que compara el promedio de beneficios físicos/offline frente a plataformas de streaming, agrupados por el género/perfil de audiencia de cada artista en tus mock data.
- **Barra Lateral Derecha (New Releases):** Presenta un álbum destacado (como *Proof* de BTS) acompañado por tres mini discos de vinilo rojos superpuestos que giran suavemente. Abajo, se muestra una lista vertical interactiva de las canciones del momento ordenadas cronológicamente, con barras de desplazamiento personalizadas con degradados de neón.

4.

5. **Bitácora de Desarrollo de IA (AI Tools Usage Log):**

- He creado un archivo `README.md` completo en la raíz del proyecto para cumplir con los requisitos rigurosos de tu examen técnico para el rol de **Senior Frontend Engineer**, incluyendo la sección requerida sobre cómo el agente y tú colaboraron utilizando herramientas de IA.

6.

---

## Estructura de Archivos Creados y Modificados

Para mantener un código ultra limpio y evitar problemas de límite de tokens, el proyecto se ha modularizado con precisión profesional:

- `/src/data/artists.ts`: Estructura de datos e interfaces de Artistas.
- `/src/data/albums.ts`: Estructura de datos e interfaces de Álbumes.
- `/src/data/songs.ts`: Estructura de datos e interfaces de Canciones.
- `/src/components/WelcomeScreen.tsx`: Pantalla de bienvenida con los tres vinilos animados.
- `/src/components/LeftMenu.tsx`: Menú lateral izquierdo en cápsula de cristal.
- `/src/components/SearchBar.tsx`: Buscador interactivo y reactivo de contenido.

- `/src/components/TopCloudAlbums.tsx`: Estructuración tipográfica del logotipo "cloud" con portadas de álbumes.
- `/src/components/TopArtists.tsx`: Fila de avatares de los 5 artistas principales.
- `/src/components/AnalyticsCharts.tsx`: Implementación interactiva de Recharts de neón en tiempo real.
- `/src/components/NewReleases.tsx`: Barra lateral derecha con canciones de lanzamiento y vinilos de color rojo.
- `/src/index.css`: Carga de fuentes (Inter, Space Grotesk, JetBrains Mono) y utilidades de cristal templado y neón de Tailwind.
- `/src/App.tsx`: Ensamblado e interacción global del Dashboard.
- `/README.md`: Bitácora de desarrollo y explicación técnica detallada del proyecto.

### ¿Lo usaste tal cual o lo modificaste?

Lo modifique

### ¿Por qué tomaste esas decisiones?

El código contenía problemas y errores. Además edite y desagregue algunos componentes que me dio la IA, para más fiabilidad a mi diseño

### **\*\*Aprendizaje:\*\***

#### ¿Qué aprendiste de esta interacción?

-Reglas de diseño específicas, como el efecto glass

¿Haría algo de manera diferente?

---

## 3- Interacción significativa con la IA

### [Artist profile View]

**Objetivo** Desarrollar la pantalla dinámica de detalle de artista en Next.js 16+ y Tailwind CSS, con elementos fijos y contenido scrolleable, que renderice en tiempo real los KPIs financieros, el alcance global en mapa de calor, el banner de álbum/sencillo y los ingresos por categoría

**Herramienta de IA utilizada:** Google studio IA

### **Prompt/Approach:**

- Que le preguntaste?

Ya implementamos el Landing Section y el Main Dashboard View. Ahora debes implementar la Pantalla de Detalle de Artista (ArtistView), la cual se activa de forma dinámica cuando el usuario busca y selecciona a cualquiera de los 9 artistas presentes en nuestra mock data

---

## Requisitos

### 1. Selección:

- Debe funcionar de manera fluida para los 9 artistas del dataset (Sabrina Carpenter, Ariana Grande, Bruno Mars, Bad Bunny, Tyler, The Creator, Olivia Rodrigo, Dua Lipa, BTS, Lady Gaga).

### 2. Elementos Estáticos:

- Buscador Superior: Permanece fijo en la parte central superior.
- Foto del Artista: Imagen a tamaño completo (artistPhoto) ubicada en el lateral derecho de la página, fija en pantalla.
- Sidebar Permanece fijo a la izquierda. Incluye:
  - Submenú de vistas: Artista, Canciones, Álbumes.
  - Lista de artistas agregados a Favoritos.
  - Lista de artistas vistos Recientemente.
  - Selector/Símbolo para cambio de idioma

---

## # Layout y Contenido Central (Sección Dynamic Scroll)

La sección central del artista debe maquetarse respetando exactamente la siguiente jerarquía visual:

### 1. Cabecera del Artista y KPIs Principales

- Nombre del Artista: \*Estilizado en tipografía Inter Light Italic.
- KPIs (Fila a la derecha del nombre):
  - Conversion Rate: % de oyentes mensuales que compran entradas o merch
  - Subscription Earnings: Ingreso mensual obtenido por membresías/fan club oficial.
  - Net Benefits: Beneficio neto obtenido en el último lanzamiento de álbum.

### 2. Banner de Álbum / Canción Destacada

- Contenedor con efecto tipo nota/card que consume la propiedad `bannerSong` del artista.
- Muestra: Portada del álbum, recuento acumulado de likes/interacciones, una breve descripción y desglose visual de interacciones (YouTube likes, Spotify playlist saves, Apple Music likes).

### 3. Analytics y Desglose Financiero (Zona Derecha)

- Gráfico de Ingresos por Categoría (Últimos 3 años): \*\*
  - Gráfico interactivo (BarChart o Donut/PieChart con Recharts) que desglose los ingresos del artista consumiendo la propiedad `graphics`: merchEarning, `showsEarning` (giras), `albumEarning` y `streamingEarning`.
- Top Canciones más Rentables (Widget inferior al gráfico):
  - Lista vertical que muestra las canciones más rentables del artista (profitableTracks), combinando ingresos de streaming y descargas.

### 4. Alcance Global (Zona Izquierda Inferior)

- Gráfico de Mapa de Calor:
  - Visualización geográfica (Mapamundi vectorial/SVG o representación tipo Heatmap) con puntos de luz/neón que representan los países donde el artista tiene mayor impacto y consumo (`globalReach`).

---

## Reglas

- Asegura que al cambiar de artista, todo el estado de la página (KPIs, gráficos, fotos y notas) se actualice instantáneamente sin recargar la aplicación.



- How did you frame the problem?

El objetivo central fue construir una vista de perfil de artista donde el estado elegido (el artista seleccionado entre los 9 disponibles) controle instantáneamente múltiples widgets, métricas y elementos visuales, mientras mantiene elementos clave del layout completamente estáticos. Se identificó que la pantalla requiere una composición de dos capas: elementos estáticos (Sidebar izquierdo, Buscador superior y Foto del artista en el lateral derecho) y una zona central scrolleable (KPIs, Banner, Gráficos de categoría, Mapa de calor y Tracks rentables).

## **\*\*Resultados:\*\***

### **-Que te dio la IA?**

He completado con éxito la implementación de la nueva **pantalla detallada de Artistas**, logrando un diseño de altísima fidelidad visual y comportamiento interactivo continuo que replica exactamente el wireframe y los requerimientos especificados.

Aquí tienes un resumen de las mejoras, estructuras de diseño y características implementadas:

## **Concepto y Arquitectura de la Interfaz**

- **Estructura Estática Tridireccional:** Para lograr la perfecta inmovilidad de los elementos clave solicitados durante el desplazamiento (`scrolling`), se diseñó un contenedor con las siguientes propiedades:
  - **Menú Lateral Estático (Izquierda):** Un panel alargado con diseño de cápsula de vidrio flotante que se mantiene fijo a la izquierda, permitiendo navegar libremente.
  - **Buscador Estático (Centro Superior):** Ubicado de forma fija mediante un fondo desenfocado translúcido (`sticky bg-black/80 backdrop-blur-md`) para que el usuario pueda filtrar canciones en todo momento.



- **Retrato del Artista Estático (Derecha):** Un imponente póster vertical del artista que permanece estático en el costado derecho de la pantalla (*fixed*), difuminándose sutilmente en los bordes mediante gradientes lineales y radiales oscuros que se integran con elegancia absoluta en el fondo del lienzo.
- **Cuerpo Central Desplazable:** La columna central donde residen la información y los gráficos tiene un desplazamiento suave, manteniendo los KPIs y las métricas en perfecta legibilidad.

---

## Módulos Funcionales Implementados

### 1. Nombre en Inter Light Italic y KPIs Clave (Español):

- El nombre del artista se renderiza en una tipografía de gran escala con estilos ligeros e inclinados (`font-sans font-light italic text-5xl md:text-7xl`).
- A la derecha se colocan tres tarjetas circulares oscuras con bordes de alto contraste que muestran las métricas en español: **Conversión Oyentes**, **Ingresos por suscripciones**, y el **Beneficio Neto** de su último lanzamiento.

2.

### 3. Nota Destacada "Last Album":

- Diseñada como una tarjeta de vidrio flotante con la portada del álbum más famoso del artista en la esquina superior derecha.
- Muestra el título de la canción insignia (ej. *"Espresso"*) en tipografía blanca y cursiva de gran grosor.
- Desglosa cuantitativamente el recuento de interacciones de los fanáticos divididos por plataformas: **Me Gusta en YouTube**, **Saves en Spotify** y **Likes en Apple Music**, acompañado de una breve descripción conceptual del álbum.

4.

### 5. Gráfico de Flujo de Ingresos de 3 Años:

- Un gráfico de áreas apiladas interactivo y de aspecto futurista con curvas de colores neón brillante (púrpura y naranja) construido con **Recharts**.
- Desglosa la facturación histórica (2024–2026) en las categorías especificadas: **Giras/Shows**, **Streaming**, **Álbumes** y **Merch** para cada uno de los 9 artistas.

6.

### 7. Mapa de Calor Mundial (World Audience Reach):

- Creado con un mapamundi de rejilla de puntos vectoriales estilizados.

- Genera de forma dinámica luces de neón parpadeantes (`animate-ping`) en las coordenadas geográficas de los países donde el artista tiene mayor impacto según su arreglo `globalReach`.

8.

9. **Lista de Canciones más Rentables (Top Profitable Tracks):**

- Un componente deslizable en la parte inferior derecha que ordena las canciones de mayor a menor rentabilidad económica (monetización de streams + descargas directas) con un indicador visual de barra de progreso que emula los controles del wireframe original.

10.

11. **Menú Lateral Multi-Navegación:**

- **Library (Biblioteca):** Permite alternar entre la vista detallada del **Artista**, la pestaña de análisis detallado de sus **Canciones** y el rendimiento financiero/ROI de sus **Álbumes**.
- **My favs (Favoritos):** El usuario puede marcar como favoritos a los artistas haciendo clic en el corazón, lo que añade de inmediato el artista al menú para un acceso directo.
- **Recently (Recientes):** Un motor que rastrea automáticamente los últimos artistas que el usuario ha visitado para permitirle regresar a ellos con un solo clic.
- **Selector de Idioma Dinámico (Botón de Globo):** Al hacer clic en el botón del globo terráqueo en la base del menú, se despliega un panel que permite alternar el tablero instantáneamente entre **Español** e **Inglés**, traduciendo todos los títulos, métricas y leyendas del sistema en tiempo real.

¿Lo usaste tal cual o lo modificaste? Use it as-is

Use [as.it](https://as.it), solo modifique unas librerías que no eran compatibles con el código

¿Por qué tomaste esas decisiones? Por problemas en la estructura del código

**\*\*Aprendizaje:\*\***

¿Qué aprendiste de esta interacción?

¿Haría algo de manera diferente?

---

## 4- Interacción significativa con la IA

[Album Profile View]

Objetivo Diseñar e implementar una **Vista de Álbum** que mantenga la estructura de componentes existente (Searchbar, Sidebar) mientras muestra métricas clave (ROI, Punto de

Equilibrio, Completion Rate), rendimiento de canciones y gráficos interactivos, todo respaldado por una estructura de mock Data actualizada

**Herramienta de IA utilizada:** Google studio IA

**Prompt/Approach:**

**- Que le preguntaste?**

Vamos a construir la pantalla de detalle de álbum ("Album View").

Cuando un usuario busca y selecciona un álbum desde el buscador, será redirigido a esta vista. A continuación te detallo la estructura, los componentes y los requerimientos de datos.

#### 1. Base

- Mantén la misma barra de navegación superior (Search Bar) y el menú lateral (Sidebar) exactamente igual a como están implementados en el resto del proyecto.

#### 2. Estructura de la Pantalla

- Título: Justo debajo del panel de búsqueda, muestra el nombre del álbum en tamaño grande utilizando la tipografía Inter.

- Fila de KPIs (3 Tarjetas):

1. ROI Alcanzado: Retorno de inversión del álbum en porcentaje/monto.

2. Punto de Equilibrio (Break-Even Point): Unidades o reproducciones necesarias para recuperar la inversión de producción.

3. Album Completion Rate: Porcentaje de oyentes que escuchan el álbum completo (de primera a última canción) en lugar de solo el single.

--

#### 3. Sección Media (Canción Destacada + Tracklist + Disco Giratorio)

Crea una distribución en columnas para esta sección:

- Columna Izquierda (Canción Top):

- Tarjeta de la canción más reproducida del álbum (reutiliza exactamente el mismo componente/diseño de tarjeta usado en la pantalla de "Artist").

- Columna Central (Tracklist):

- Lista interactiva del resto de las canciones del álbum, mostrando título, número de lista y su conteo de reproducciones/visualizaciones.

- **\*\*Columna Derecha / Elemento Decorativo:\*\***

- Renderiza la imagen PNG del disco/vinilo del álbum.

- Aplica una animación CSS continua de rotación (spin) para simular que el disco se está reproduciendo.

---

#### 4. Sección Inferior (Gráficos y Visualización)

Ubica dos gráficos uno al lado del otro al final de la página:

##### 1. Indicador de Punto de Equilibrio (Break-Even Progress Gauge):

- Tipo: Gráfico de arco / semicircular (Gauge Chart) con marcador de objetivo.

- Lógica visual: Muestra el progreso de 0% a 100%+.

- Si está por debajo del 100%, muestra en tono neutro/alerta el monto restante por recuperar (ej. "Faltan \$1,200 para recuperar la inversión").

- Si supera el 100%, cambia a color verde/neón destacando el ROI positivo alcanzado.

##### 2. Matriz de Combinación de Ingresos (Revenue Stream Mix):

- Tipo: Gráfico de Donut (Donut Chart) minimalista.

- Desglose de datos:

- Streaming directo de canciones del álbum.

- Venta física (Vinilos y CDs).

- Merchandising oficial asociativo del álbum.

- Licencias y Sincronización (Cine, TV, Videojuegos).

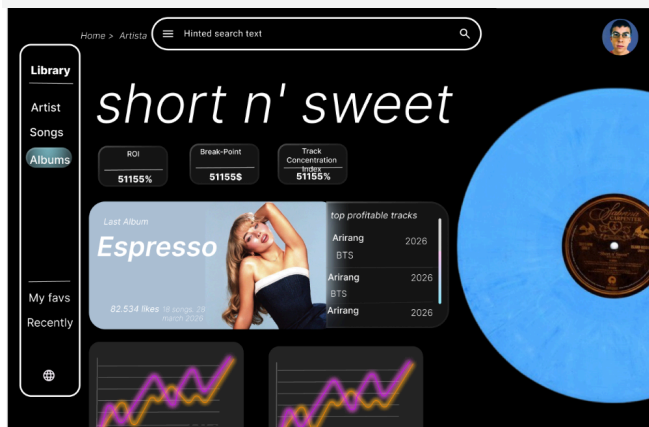
---

#### 5. Requerimiento de Mock Data

Actualiza la estructura del archivo de datos de prueba (`mockData`) para dar soporte a todos los elementos descritos arriba. Incluye los campos necesarios para:

- Inversión total vs. ingresos generados (para calcular ROI y Break-Even).
- Progreso de Break-Even (\$ restante o superado).
- Porcentaje de Album Completion Rate.
- Arreglo de distribución de ingresos por fuente (Streaming, Físico, Merch, Licencias).
- URL de la imagen en PNG del disco/vinilo.

Genera el código modular y limpio manteniendo la coherencia estética del proyecto.



- How did you frame the problem?

El objetivo en esta pantalla es hacer un análisis contable y de rendimiento enfocado en el álbum. El reto clave fue realizar métricas de producción y consumo musical (ROI, punto de equilibrio, retención de escuchas) en una interfaz visualmente atractiva e interactiva

**Resultados:**

**-Que te dio la IA?**

I have successfully completed and verified the **Music Album Detail Screen** for the EVEN Artist Dashboard.

Here is a summary of the design elements and interactive modules implemented:

## 1. Aesthetic Pairings & Typographic Hierarchy

- **Inter Display Title:** The album name is showcased in a grand, lowercase `font-sans font-light italic` layout, capturing the high-contrast editorial look requested.
- **Dynamic Breadcrumbs:** Automatically reflects the navigation path (`Inicio > Álbumes > [Album Name]`), allowing seamless tracking and backward routing.

- **Responsive Layout Collapsing:** When viewing an album detail, the central panel automatically expands to 10 out of 12 columns to comfortably host the rich visualizations, gracefully hiding the default sidebar portrait.

## 2. High-Contrast Interactive KPIs

Styled in sleek glassmorphism panels mimicking the mockups:

- **ROI Alcanzado:** Displays the precise percentage of return on production investments.
- **Break-Point:** Displays the exact target threshold in units and USD required to recover costs.
- **Album Completion Rate:** Represents the percentage of users listening to the full album from start to finish.

## 3. Track Spotlight & Tracklist Modules

- **Most Played Single:** A prominent spotlight glass card styled with glowing radial backdrops, detailing total likes and stream breakdowns per platform (Spotify, YouTube, Apple Music) for the album's lead single.
- **Tracks Index:** A scrollable side tracklist featuring all other tracks in the album with their respective total view counts.

## 4. Advanced Visualizations

- **Break-Even Progress Gauge:** A glowing, semicircular progress gauge (SVG-drawn for pixel-perfect fluidity). If the album is below its break-even point, it shifts to a coral-red warning tone and highlights the exact dollar amount remaining to recover costs. Once profitability is achieved, it lights up in neon emerald green.
- **Revenue Stream Mix Matrix:** A centered donut chart (using `recharts`) showing the proportion of digital streams, vinyl/CD physical sales, official merchandising, and sync licensing.

## 5. Spinning Vinyl Record Showcase

- **Infinite Rotation:** On the right side, a high-fidelity rendering of a physical vinyl record spins infinitely.
- **Artist Themes:** Each album has a custom-themed disc color (e.g., Sky Blue for Sabrina Carpenter's *Short n' Sweet*, warm Peach for Ariana Grande, Emerald Green for Tyler, etc.) accented with concentric sound-groove ridges, a center artwork label sticker, and a static metallic tonearm stylus needle indicating active playback.

## 6. Seamless Navigation Routing

- **Search bar redirect:** Searching for any of the 9 albums (or individual songs) in the search bar and selecting them now routes directly to their detailed Album page instead of a generic artist view.
- **Albums List Integration:** Clicking on any album card inside the standard *Albums* tab of an artist now smoothly transitions into its dedicated spotlight screen.

¿Lo usaste tal cual o lo modificaste?

Lo modifique

¿Por qué tomaste esas decisiones?

Aunque la IA implementó con éxito la estructura principal y los módulos visuales de la pantalla, omitió un elemento funcional: no generó el botón de interacción/navegación necesario para acceder a la vista detallada del álbum desde la pantalla previa.

**\*\*Aprendizaje:\*\***

¿Qué aprendiste de esta interacción?

¿Haría algo de manera diferente?

---

## 5- Interacción significativa con la IA

### [Songs Profile View]

**Objetivo** Diseñar e implementar una pantalla de detalle de Canción, que traduzca métricas complejas de retención auditiva, ingresos por plataforma y compromiso del fan (*Radar Chart*), y actualizar el esquema de mockData del proyecto

**Herramienta de IA utilizada:** Google IA studio

**Prompt/Approach:**

- Que le preguntaste?

Vamos a construir la pantalla de detalle de canción ("Song View"), tomando como referencia la arquitectura visual de la pantalla de álbumes.

Cuando un usuario busca y selecciona una canción desde el buscador, será redirigido a esta vista. A continuación te detallo la estructura, la lógica de diseño y los requerimientos de datos.

---

1. Base

- Mantén fijos el panel de búsqueda superior (SearchBar) y el menú lateral (Sidebar), exactamente como en el resto de las vistas del dashboard.

## 2. Banner Principal de la Canción

Ubica en la parte superior (justo debajo del buscador) una tarjeta de ancho completo con la siguiente configuración:

- Lado Izquierdo:

- Título de la canción en tipografía destacada de gran tamaño Inter.

- Desglose de métricas de visualizaciones, reproducciones y likes por plataforma (Spotify, YouTube, Apple Music, etc.).

- Lado Derecho:

- Carátula/portada del álbum al que pertenece la canción.

- Estilo Dinámico:

- El fondo de la tarjeta debe adaptar su color/gradiente en función del color predominante de la portada del álbum.

## 3. Fila de KPIs (4 Tarjetas con Borde Neón)

Justo debajo del banner principal, presenta 4 tarjetas de métricas individuales. Cada una debe estar enmarcada con un borde con brillo neón:

1. Save Rate: Porcentaje de oyentes que guardan la canción en su biblioteca.

2. UGC (User Generated Content): Impacto y uso del audio en contenido de usuarios (TikTok, Reels, Shorts).

3. ePMR (Effective Pay Per Mille): Tasa efectiva de ganancias por cada 1,000 reproducciones.

4. Track-to-Profile Rate: Ratio de conversión de oyentes casuales a visitas directas al perfil del artista.

## 4. Sección Inferior (Gráficos Avanzados)



Renderiza tres gráficos interactivos con las siguientes especificaciones:

### 1. Evolución de Ingresos por Plataforma:

- Tipo: Gráfico de Líneas (Line Chart).
- Objetivo: Mostrar las ganancias generadas por la canción a lo largo del tiempo, desglosadas por cada plataforma de streaming.

### 2. Curva de Retención y Abandono (Audio Retention Chart):

- Tipo: Gráfico de Área o Línea continua.
- Ejes: Eje X (Tiempo exacto de la canción en min:seg), Eje Y (Porcentaje de oyentes activos del 0% al 100%).
- Objetivo: Visualizar en qué segundo exacto de la canción los usuarios dejan de escuchar o saltan el track.

### 3. Radar de Compromiso del Fan:

- Tipo: Gráfico de Araña / Radar (Spider/Radar Chart).
- Vértices (5 Ejes):
  - Completion Rate
  - Save-to-Stream Ratio
  - Inclusión en Playlists de Usuarios
  - Tasa de Compartidos (Shares)
  - Ingreso por Stream (ePMR)

---

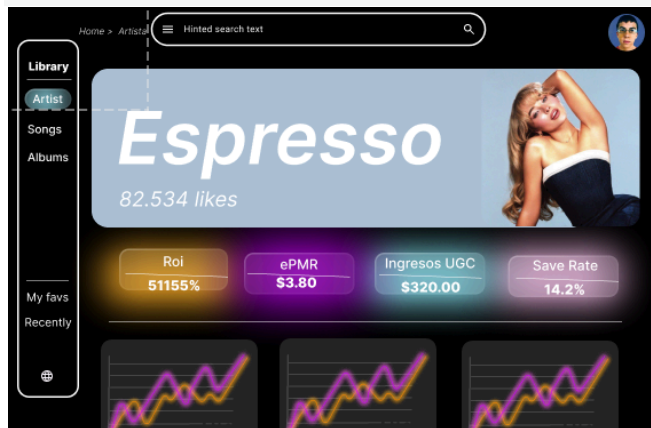
### 5. Actualización de Mock Data

Modifica la estructura de mockData para alimentar esta pantalla de forma reactiva, incluyendo:

- Valores numéricos para los 4 KPIs de conversión/engagement.
- Series de tiempo para ingresos por plataforma.
- Array de puntos de retención (`timestamp` vs `% retención`).

- Objeto de métricas para los 5 vértices del Radar Chart.

Genera el código modular y limpio manteniendo la coherencia estética del proyecto.



### - How did you frame the problem?

El objetivo en esta pantalla es traducir el consumo de canciones en analíticas visuales: no solo mostrar cuántas reproducciones tiene la canción, sino entender dónde genera dinero (plataformas), cómo la escuchan (curva de retención) y qué tanto engancha al usuario (radar de engagement y viralidad UGC), todo sin perder la continuidad estética.

### Resultados:

#### -Que te dio la IA?

He realizado la implementación completa de la **Pantalla de Canciones** siguiendo la estructura visual y especificaciones solicitadas:

- **Tarjeta Principal de Canción (Hero Banner):**
  - **Fondo Dinámico:** Utiliza el color predominante de la portada del álbum (`cardColor`) con degradados suaves para asegurar legibilidad.
  - **Título del Tema:** Nombre de la canción en tipografía *italic* de gran formato.
  - **Portada a la Derecha:** Imagen del álbum ubicada en el lateral derecho de la tarjeta con opción de reproducción en hover.
  - **Métricas de Vistas y Me Gusta:** Conteo formateado de likes y desglose por plataformas (*Spotify, YouTube, UGC*).
  - **Selector de Canción:** Desplegable rápido para cambiar la canción activa desde la tarjeta.
- 
- **KPIs con Brillo Neón:**
  - **Save Rate:** Marco con aura resplandeciente en tono dorado/ámbar.
  - **UGC (User Generated Content):** Recuadro con brillo púrpura/violeta (ingresos y recuento de videos).

- **ePMR (Pay Per Mille)**: Indicador en tono cian/turquesa neón.
- **Track-to-Profile Rate**: Tasa de conversión a fan directo en tono neón rosa.
- 
- **Gráficos de Rendimiento**:
  - **Líneas de Ganancias por Plataforma**: Evolución de ingresos en Spotify, Apple Music, YouTube y TikTok.
  - **Curva de Retención y Abandono**: Gráfico de área (Tiempo vs. % de Oyentes Activos) para analizar los puntos de enganche de la canción.
  - **Radar de Engagement (Spider Chart)**: Evalúa las 6 métricas clave (*Completion Rate, Save-to-Stream, Playlist Adds, Shares, ePMR, Fan Conversion*).
- 
- **Navegación e Integración con Buscador**:
  - Buscar cualquier canción en la barra superior estática redirige de inmediato a la vista detallada del perfil de dicha canción.

#### ¿Lo usaste tal cual o lo modificaste?

Lo modifique

#### ¿Por qué tomaste esas decisiones?

Aunque la IA generó la estructura base y los bloques solicitados, fue necesario realizar cambios estéticos: La IA generó de forma automática una imagen que rompía con la estética de la pantalla, por lo que fue retirado, también refiné los bordes neón de las tarjetas de KPI y los achique

#### Aprendizaje:

¿Qué aprendiste de esta interacción?

¿Haría algo de manera diferente?

---



---

## PREGUNTAS DE REFLEXIÓN

**\*a) Estrategia de IA:**

### ¿Cómo decidiste cuándo usar IA y cuándo programar desde cero?

Siendo sincera, decidí el uso de la IA para hacer la base robusta de todo el proyecto. El desarrollo de código lo reservé para corregir inconsistencias visuales respecto a la idea original y resolver errores de lógica, tipado o compatibilidad donde la IA no lograba la precisión requerida

### b) ¿Hubo alguna tarea en la que la IA no fue útil? ¿Por qué?

Sí. Al integrar la Google AI Studio, surgió un conflicto de compatibilidad con una librería. Para resolverlo, intenté apoyarme en la IA de Cursor, pero la sugerencia empeoró el panorama generando más de 300 errores en el proyecto. En ese punto, la IA dejó de ser útil; decidí hacer un *rollback* de los cambios e investigar de forma autónoma la documentación para solucionar el conflicto por mi cuenta.

## **b) Autoría y Dominio del Código (Code Ownership):**

### **-Cómo te aseguraste de entender todo el código generado por IA?**

Mantuve un proceso de revisión activa: Le pedí a la misma IA que me explicara lo realizado, haciendo mas detalle en las secciones de código que le subrayaba

### **-Describe un fragmento de código generado por IA que hayas modificado significativamente y explica por qué.**

El fragmento de código que mas modifique fue el de "songsoverview", ya que pude observar que había un conflicto al momento del filtrado. Cuando el usuario cambiaba el artista en el filtro, el estado `selectedAlbumFilter` se reinicia a "all". Pero si volvía a cambiar a "Todos los Artistas", el álbum seleccionado previamente no se limpia automáticamente, generando desincronización, así que lo arregle. Además de eso, quite algunas funciones para que el filtro se viera mas estetico

## **c) Impacto en la Productividad (Productivity Impact):**

### **-Estima cuánto tiempo te ahorró (o no) la IA.**

Ahorré una cantidad significativa de tiempo, especialmente al usar herramientas con las cuales pude ir viendo cómo se iba maquetando la página según lo que agregaba.

### **-¿Qué habrías hecho de manera diferente sin el uso de herramientas de IA?**

Sin la IA, habría replicado mi maquetado inicial de Figma. Sin embargo, considero que el uso de estas herramientas me permitió lograr un resultado estético más pulido y funcional.

## **d) Control de Calidad (Quality Assurance):**

### **¿Cómo validaste el código generado por la IA?**

Mediante pruebas funcionales continuas en el entorno local (`localhost`), verificando que el comportamiento de los componentes, el flujo de estado y la respuesta de la UI coincidieran exactamente con las especificaciones.

### **¿Detectaste algún error o problema en las sugerencias de la IA?**

Sí, con frecuencia. La mayoría eran problemas de tipado e incompatibilidad de interfaces en TypeScript, como diferencias entre IDs numéricos y cadenas, o propiedades opcionales no contempladas en las props de los componentes

## **3. Un Ejemplo Detallado (One Detailed Example)**

Elige **UNA** funcionalidad o componente significativo y realiza un análisis profundo:

- **Muestra el prompt inicial.**

Vamos a construir la pantalla intermedia de "Catálogo de Canciones", la cual sirve como paso previo antes de ingresar al perfil detallado de una canción ("Song Profile View").

Esta vista debe permitir al usuario explorar, filtrar y seleccionar cualquier canción de un álbum para analizarla.

Debe tener una barra de Filtros:

- Select de Artista: Desplegable para filtrar por artista (ej. "Sabrina Carpenter").
  - Select de Álbum: Desplegable para filtrar por álbum ("Todos los Álbumes" o seleccionar uno específico).
  - Select de Ordenamiento: Desplegable "Ordenar por:" con opciones como "Más Vistas", "Más Likes", "Mayor ePMR", etc.
  - Configura los botones "Ver Perfil de Canción" para que transmitan el ID de la canción seleccionada hacia la vista de detalle.
- **Muestra las iteraciones (si las hubo).**
  - **Incluye fragmentos de código que muestren el *antes* y *después* si modificaste la respuesta de la IA.**

#### ANTES:

```
import { useState, useMemo } from "react";

import { motion } from "motion/react";

import { Song, songs } from "../data/songs";

import { Album, albums } from "../data/albums";

import { Artist, artists } from "../data/artists";

import { Music, Play, ChevronRight, Filter, Sparkles, TrendingUp, Eye,
Bookmark, Zap, Disc } from "lucide-react";

interface SongsOverviewProps {

  onSelectSong: (songId: number) => void;

  onPlaySong?: (song: Song) => void;

  language?: "es" | "en";

  currentArtistId?: number;

}
```

```

export default function SongsOverview({

  onSelectSong,

  onPlaySong,

  language = "es",

  currentArtistId,

}: SongsOverviewProps) {

  // State filters

  const [selectedArtistFilter, setSelectedArtistFilter] = useState<number |
  "all">(currentArtistId || "all");

  const [selectedAlbumFilter, setSelectedAlbumFilter] = useState<number |
  "all">("all");

  const [sortBy, setSortBy] = useState<"views" | "saveRate" | "ugc" |
  "epmr">("views");

  // Translations

  const t = {

    es: {

      title: "Catálogo de Canciones",

      subtitle: "Explora y selecciona cualquier canción por álbum para
      analizar su retención, engagement y rendimiento en plataformas.",

      filterArtist: "Artista:",

      filterAlbum: "Álbum:",

      sortBy: "Ordenar por:",

      allArtists: "Todos los Artistas",

      allAlbums: "Todos los Álbumes",

      views: "Más Vistas",

      saveRate: "Mayor Save Rate",

```

```
    ugc: "Mayor UGC",

    epmr: "Mayor ePMR",

    tracksInAlbum: "canciones en este álbum",

    viewProfile: "Ver Perfil de Canción",

    saveRateLabel: "Save Rate",

    ugcLabel: "Videos UGC",

    epmrLabel: "ePMR",

    totalViewsLabel: "Vistas Totales",

  },

  en: {

    title: "Songs Catalog",

    subtitle: "Browse and select any track by album to analyze retention,
engagement, and platform performance.",

    filterArtist: "Artist:",

    filterAlbum: "Album:",

    sortBy: "Sort by:",

    allArtists: "All Artists",

    allAlbums: "All Albums",

    views: "Most Views",

    saveRate: "Highest Save Rate",

    ugc: "Highest UGC",

    epmr: "Highest ePMR",

    tracksInAlbum: "tracks in this album",

    viewProfile: "View Song Profile",

    saveRateLabel: "Save Rate",

    ugcLabel: "UGC Videos",
```

```

        epmrLabel: "ePMR",

        totalViewsLabel: "Total Views",

    },

}[language];

// Filtered albums list based on selected artist filter

const filteredAlbums = useMemo(() => {

    let result = albums;

    if (selectedArtistFilter !== "all") {

        result = result.filter((al) => al.artistId === selectedArtistFilter);

    }

    if (selectedAlbumFilter !== "all") {

        result = result.filter((al) => al.id === selectedAlbumFilter);

    }

    return result;

}, [selectedArtistFilter, selectedAlbumFilter]);

// Sort songs helper

const sortSongsList = (list: Song[]) => {

    return [...list].sort((a, b) => {

        if (sortBy === "views") return b.totalViews - a.totalViews;

        if (sortBy === "saveRate") return b.saveRate - a.saveRate;

        if (sortBy === "ugc") return b.ugcRevenue - a.ugcRevenue;

        if (sortBy === "epmr") return b.epmr - a.epmr;

        return 0;

    });

};

```



```

};

return (

  <div className="w-full space-y-6 pb-16" id="songs-overview-container">

    { /* 1. HEADER BANNER */ }

    <div className="rounded-3xl border border-white/10 glass-panel p-6
shadow-2xl space-y-4 relative overflow-hidden">

      <div className="absolute -top-24 -right-24 w-60 h-60 bg-pink-500/10
rounded-full blur-3xl pointer-events-none" />

      <div className="absolute -bottom-24 -left-24 w-60 h-60
bg-purple-500/10 rounded-full blur-3xl pointer-events-none" />

      <div className="flex flex-col md:flex-row md:items-center
justify-between gap-4 relative z-10">

        <div className="space-y-1">

          <div className="flex items-center gap-2">

            <div className="w-8 h-8 rounded-full bg-pink-500/10 flex
items-center justify-center border border-pink-500/20">

              <Music className="w-4 h-4 text-pink-500" />

            </div>

            <h1 className="text-xl sm:text-2xl font-black text-white
uppercase font-mono tracking-wide">

              {t.title}

            </h1>

          </div>

          <p className="text-xs text-neutral-400 max-w-xl font-sans">

            {t.subtitle}


```

```

    </p>

</div>

    <div className="flex items-center gap-3 font-mono text-xs
text-white/80 bg-black/40 px-3.5 py-2 rounded-2xl border border-white/10
self-start md:self-auto">

        <Sparkles className="w-4 h-4 text-amber-400 animate-pulse" />

        <span>{songs.length} Canciones Disponibles</span>

    </div>

</div>

    { /* FILTERS AND SORT BAR */ }

    <div className="flex flex-wrap items-center justify-between gap-3
pt-3 border-t border-white/5 relative z-10 text-xs font-mono">

        { /* Left Side: Artist and Album Dropdowns */ }

        <div className="flex flex-wrap items-center gap-2">

            <div className="flex items-center gap-1.5 bg-neutral-900/80 px-3
py-1.5 rounded-xl border border-white/10">

                <span className="text-neutral-500">{t.filterArtist}</span>

                <select

                    value={selectedArtistFilter}

                    onChange={ (e) => {

                        const val = e.target.value === "all" ? "all" :
Number(e.target.value);

                        setSelectedArtistFilter(val);

                        setSelectedAlbumFilter("all"); // reset album filter when
artist changes

```

```

    }}

    className="bg-transparent text-white outline-none
    cursor-pointer font-bold"

  >

    <option value="all" className="bg-neutral-900
    text-white">{t.allArtists}</option>

    {artists.map((a) => (

      <option key={a.id} value={a.id} className="bg-neutral-900
      text-white">

        {a.name}

      </option>

    ))}

  </select>

</div>

<div className="flex items-center gap-1.5 bg-neutral-900/80 px-3
py-1.5 rounded-xl border border-white/10">

  <span className="text-neutral-500">{t.filterAlbum}</span>

  <select

    value={selectedAlbumFilter}

    onChange={(e) => setSelectedAlbumFilter(e.target.value ===
    "all" ? "all" : Number(e.target.value))}

    className="bg-transparent text-white outline-none
    cursor-pointer font-bold max-w-[160px] truncate"

  >

    <option value="all" className="bg-neutral-900
    text-white">{t.allAlbums}</option>

    {(selectedArtistFilter === "all" ? albums : albums.filter(al
    => al.artistId === selectedArtistFilter)).map((al) => (

```

```
        <option key={al.id} value={al.id}
className="bg-neutral-900 text-white">
```

```
        {al.titleAlbum}
```

```
    </option>
```

```
  )}}
```

```
</select>
```

```
</div>
```

```
</div>
```

```
    { /* Right Side: Sorting Dropdown */ }
```

```
    <div className="flex items-center gap-1.5 bg-neutral-900/80 px-3
py-1.5 rounded-xl border border-white/10 ml-auto sm:ml-0">
```

```
      <Filter className="w-3.5 h-3.5 text-pink-400" />
```

```
      <span className="text-neutral-500">{t.sortBy}</span>
```

```
      <select
```

```
        value={sortBy}
```

```
        onChange={ (e) => setSortBy(e.target.value as any) }
```

```
        className="bg-transparent text-pink-400 font-bold outline-none
cursor-pointer"
```

```
      >
```

```
        <option value="views" className="bg-neutral-900
text-white">{t.views}</option>
```

```
        <option value="saveRate" className="bg-neutral-900
text-white">{t.saveRate}</option>
```

```
        <option value="ugc" className="bg-neutral-900
text-white">{t.ugc}</option>
```

```
        <option value="epmr" className="bg-neutral-900
text-white">{t.epmr}</option>
```

```
      </select>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* 2. ALBUMS AND SONGS CATALOG GRID */}
```

```
<div className="space-y-8">
```

```
  {filteredAlbums.map((album) => {
```

```
    const albumArtist = artists.find((a) => a.id === album.artistId)
    || artists[0];
```

```
    const albumSongs = songs.filter((s) => s.albumId === album.id);
```

```
    const sortedSongs = sortSongsList(albumSongs);
```

```
    if (sortedSongs.length === 0) return null;
```

```
    return (
```

```
      <motion.div
```

```
        key={album.id}
```

```
        initial={{ opacity: 0, y: 12 }}
```

```
        animate={{ opacity: 1, y: 0 }}
```

```
        transition={{ duration: 0.4 }}
```

```
        className="rounded-3xl border border-white/10 glass-panel p-6
shadow-xl space-y-5 relative overflow-hidden"
```

```
      >
```

```
        {/* ALBUM HEADER SECTION */}
```

```
        <div className="flex flex-wrap items-center justify-between
gap-4 border-b border-white/10 pb-4">
```

```

<div className="flex items-center gap-4">

  <div className="relative w-14 h-14 rounded-2xl
overflow-hidden border border-white/20 shadow-md flex-shrink-0">

    <img src={album.coverUrl} alt={album.titleAlbum}
className="w-full h-full object-cover" />

  </div>

  <div>

    <div className="flex items-center gap-2">

      <span className="text-xs font-bold text-pink-400
uppercase tracking-widest font-mono">

        {albumArtist.name}

      </span>

      <span className="text-[10px] text-neutral-500
font-mono">&bull; {album.year}</span>

    </div>

    <h2 className="text-lg font-bold text-white
tracking-tight font-sans">

      {album.titleAlbum}

    </h2>

  </div>

</div>

<div className="flex items-center gap-2 font-mono text-xs
text-neutral-400">

  <Disc className="w-4 h-4 text-purple-400" />

  <span>{albumSongs.length} {t.tracksInAlbum}</span>

</div>

</div>

```

```

    { /* SONGS GRID FOR THIS ALBUM */ }

    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">

      {sortedSongs.map((song) => (

        <motion.div

          key={song.id}

          whileHover={{ scale: 1.01, y: -2 }}

          transition={{ duration: 0.2 }}

          onClick={() => onSelectSong(song.id)}

          className="group relative rounded-2xl border
border-white/10 bg-neutral-900/60 hover:bg-neutral-900/95 p-4 transition
duration-300 cursor-pointer shadow-md hover:shadow-2xl overflow-hidden flex
flex-col justify-between"

          style={{

            borderColor: `${song.cardColor}40`,

          }}

        >

          { /* Hover Glow Effect using card color */ }

          <div

            className="absolute inset-0 opacity-0
group-hover:opacity-15 transition duration-500 pointer-events-none"

            style={{ backgroundColor: song.cardColor }}

          />

          { /* TOP HALF: Song Info & Artwork */ }

          <div className="flex items-start gap-3.5 relative z-10">

```

```

<div className="relative w-16 h-16 rounded-xl
overflow-hidden border border-white/15 flex-shrink-0
group-hover:border-white/40 transition">

    <img src={song.coverUrl} alt={song.titleSong}
className="w-full h-full object-cover" />

    { /* Play button overlay */ }

    <button

        onClick={ (e) => {

            e.stopPropagation();

            if (onPlaySong) onPlaySong(song);

        }}

        className="absolute inset-0 bg-black/40 opacity-0
group-hover:opacity-100 transition flex items-center justify-center"

    >

        <div className="w-8 h-8 rounded-full bg-white
text-black flex items-center justify-center shadow">

            <Play className="w-4 h-4 fill-black
translate-x-0.5" />

        </div>

    </button>

</div>

<div className="flex-1 min-w-0 space-y-1">

    <div className="flex items-center justify-between
gap-2">

        <h3 className="text-base font-bold text-white
truncate group-hover:text-pink-300 transition font-sans">

            {song.titleSong}

        </h3>

```



```
        <span className="text-[10px] font-mono font-bold
text-neutral-400 bg-black/40 px-2 py-0.5 rounded border border-white/10
flex-shrink-0">
```

```
            {song.likesFormatted} likes
```

```
        </span>
```

```
    </div>
```

```
    <div className="flex items-center gap-2 text-xs
font-mono text-neutral-400">
```

```
        <span
className="text-neutral-300">{(song.totalViews / 1000000).toFixed(0)}M
views</span>
```

```
        <span>&bull;</span>
```

```
        <span className="text-emerald-400
font-bold">${(song.kpis.earnings / 1000000).toFixed(1)}M</span>
```

```
    </div>
```

```
</div>
```

```
</div>
```

```
{/* MIDDLE: 3 MINI KPIS PILLS */}
```

```
    <div className="grid grid-cols-3 gap-2 my-3 pt-3
border-t border-white/10 relative z-10 font-mono text-[10px]">
```

```
        {/* Save Rate */}
```

```
        <div className="bg-amber-500/10 border
border-amber-500/20 rounded-xl p-2 text-center">
```

```
            <span className="text-amber-400/80 block text-[9px]
uppercase font-bold">{t.saveRateLabel}</span>
```

```
            <span className="text-xs font-extrabold
text-amber-300">{song.saveRate}%</span>
```

```
</div>
```

```
{/* UGC Videos */}
```

```
<div className="bg-purple-500/10 border  
border-purple-500/20 rounded-xl p-2 text-center">
```

```
<span className="text-purple-400/80 block text-[9px]  
uppercase font-bold">{t.ugcLabel}</span>
```

```
<span className="text-xs font-extrabold  
text-purple-300">{song.ugcVideosCount}</span>
```

```
</div>
```

```
{/* ePMR */}
```

```
<div className="bg-cyan-500/10 border  
border-cyan-500/20 rounded-xl p-2 text-center">
```

```
<span className="text-cyan-400/80 block text-[9px]  
uppercase font-bold">{t.epmrLabel}</span>
```

```
<span className="text-xs font-extrabold  
text-cyan-300">${song.epmr.toFixed(2)}</span>
```

```
</div>
```

```
</div>
```

```
{/* BOTTOM: ACTION BUTTON */}
```

```
<div className="pt-2 border-t border-white/5 flex  
items-center justify-between relative z-10">
```

```
<span className="text-[10px] font-mono  
text-neutral-500">
```

```
{albumArtist.name} &bull; {song.year}
```

```
</span>
```

```

        <div className="flex items-center gap-1 text-xs
font-mono font-bold text-pink-400 group-hover:text-pink-300 transition">

            <span>{t.viewProfile}</span>

            <ChevronRight className="w-3.5 h-3.5
group-hover:translate-x-1 transition duration-300" />

        </div>

    </div>

</motion.div>

    )}

</div>

</motion.div>

    );

    })}

</div>

</div>

    );

}

```

## DESPUÉS

```

import { useState, useMemo } from "react";
import { motion } from "framer-motion";
import { Song, songs } from "../data/songs";
import { Album, albums } from "../data/albums";

```

```

import { Artist, artists } from "../data/artists";

import { Music, Play, ChevronRight, Filter, Sparkles, Disc } from
"lucide-react";

interface SongsOverviewProps {

  onSelectSong: (songId: number) => void;

  onPlaySong?: (song: Song) => void;

  language?: "es" | "en";

  currentArtistId?: number;
}

export default function SongsOverview({

  onSelectSong,

  onPlaySong,

  language = "es",

  currentArtistId,

}: SongsOverviewProps) {

  // State filters

  const [selectedArtistFilter, setSelectedArtistFilter] =
useState<number | "all">(currentArtistId || "all");

  const [selectedAlbumFilter, setSelectedAlbumFilter] =
useState<number | "all">("all");

  const [sortBy, setSortBy] = useState<"views_desc" |
"views_asc">("views_desc");

  // Translations

  const t = {

    es: {

      title: "Catálogo de Canciones",

      subtitle: "Explora y selecciona cualquier canción por
álbum para analizar su retención, engagement y rendimiento en
plataformas.",

```

```

        filterArtist: "Artista:",
        filterAlbum: "Álbum:",
        sortBy: "Ordenar por:",
        allArtists: "Todos los Artistas",
        allAlbums: "Todos los Álbumes",
        viewsDesc: "Más Vistas",
        viewsAsc: "Menos Vistas",
        tracksInAlbum: "canciones en este álbum",
        viewProfile: "Ver Perfil de Canción",
        totalViewsLabel: "Vistas Totales",
    },
    en: {
        title: "Songs Catalog",
        subtitle: "Browse and select any track by album to
analyze retention, engagement, and platform performance.",
        filterArtist: "Artist:",
        filterAlbum: "Album:",
        sortBy: "Sort by:",
        allArtists: "All Artists",
        allAlbums: "All Albums",
        viewsDesc: "Most Views",
        viewsAsc: "Least Views",
        tracksInAlbum: "tracks in this album",
        viewProfile: "View Song Profile",
        totalViewsLabel: "Total Views",
    },
}[language];

// Options available for the album filter based ON strictly
selected artist

```

```

const availableAlbumsForFilter = useMemo(() => {
    if (selectedArtistFilter === "all") return albums;

    return albums.filter((al) => al.artistId ===
selectedArtistFilter);

}, [selectedArtistFilter]);

// Filtered albums list for rendering
const filteredAlbums = useMemo(() => {
    let result = availableAlbumsForFilter;

    if (selectedAlbumFilter !== "all") {
        result = result.filter((al) => al.id ===
selectedAlbumFilter);
    }

    return result;
}, [availableAlbumsForFilter, selectedAlbumFilter]);

// Handle Artist change safely
const handleArtistChange = (val: number | "all") => {
    setSelectedArtistFilter(val);

    setSelectedAlbumFilter("all"); // Reset album selection
to avoid empty states

};

return (
    <div className="w-full space-y-6 pb-16"
id="songs-overview-container">

        {/* 1. HEADER BANNER */}

        <div className="rounded-2xl sm:rounded-3xl border
border-white/10 glass-panel p-4 sm:p-6 shadow-2xl space-y-4
relative overflow-hidden">

            <div className="absolute -top-24 -right-24 w-60
h-60 bg-pink-500/10 rounded-full blur-3xl pointer-events-none" />

```

```

        <div className="absolute -bottom-24 -left-24 w-60
h-60 bg-purple-500/10 rounded-full blur-3xl pointer-events-none"
/>

        <div className="flex flex-col md:flex-row
md:items-center justify-between gap-4 relative z-10">

            <div className="space-y-1">

                <div className="flex items-center gap-2">

                    <div className="w-8 h-8 rounded-full
bg-pink-500/10 flex items-center justify-center border
border-pink-500/20">

                        <Music className="w-4 h-4
text-pink-500" />

                    </div>

                    <h1 className="text-xl sm:text-2xl
font-heading font-black text-white uppercase tracking-wide">

                        {t.title}

                    </h1>

                </div>

                <p className="text-xs text-neutral-400
max-w-xl font-body">

                    {t.subtitle}

                </p>

            </div>

            <div className="flex items-center gap-3
font-mono text-xs text-white/80 bg-black/40 px-3.5 py-2
rounded-2xl border border-white/10 self-start md:self-auto">

                <Sparkles className="w-4 h-4
text-amber-400 animate-pulse" />

                <span>{songs.length} Canciones
Disponibles</span>

            </div>

        </div>

```

```

        { /* FILTERS AND SORT BAR */ }

        <div className="flex flex-wrap items-center
justify-between gap-3 pt-3 border-t border-white/5 relative z-10
text-xs font-mono">

            { /* Left Side: Artist and Album Dropdowns */ }

            <div className="flex flex-wrap items-center
gap-2">

                <div className="flex items-center gap-1.5
bg-neutral-900/80 px-3 py-1.5 rounded-xl border border-white/10">

                    <span
className="text-neutral-500">{t.filterArtist}</span>

                    <select

                        value={selectedArtistFilter}

                        onChange={(e) => {

                            const val = e.target.value
                            === "all" ? "all" : Number(e.target.value);

                            handleArtistChange(val);

                        }}

                        className="bg-transparent
text-white outline-none cursor-pointer font-bold"

                    >

                        <option value="all"
className="bg-neutral-900 text-white">{t.allArtists}</option>

                        {artists.map((a) => (

                            <option key={a.id}
value={a.id} className="bg-neutral-900 text-white">

                                {a.name}

                            </option>

                        ))}

                    </select>

                </div>

```



```

        <div className="flex items-center gap-1.5
bg-neutral-900/80 px-3 py-1.5 rounded-xl border border-white/10">

            <span
className="text-neutral-500">{t.filterAlbum}</span>

            <select

                value={selectedAlbumFilter}

                onChange={(e) =>
setSelectedAlbumFilter(e.target.value === "all" ? "all" :
Number(e.target.value))}

                className="bg-transparent
text-white outline-none cursor-pointer font-bold max-w-[160px]
truncate"

                >

                    <option value="all"
className="bg-neutral-900 text-white">{t.allAlbums}</option>

                    {availableAlbumsForFilter.map((al) => (

                        <option key={al.id}
value={al.id} className="bg-neutral-900 text-white">

                            {al.titleAlbum}

                        </option>

                    ))}

                </select>

            </div>

        </div>

        { /* Right Side: Sorting Dropdown */ }

        <div className="flex items-center gap-1.5
bg-neutral-900/80 px-3 py-1.5 rounded-xl border border-white/10
ml-auto sm:ml-0">

            <Filter className="w-3.5 h-3.5
text-pink-400" />

            <span
className="text-neutral-500">{t.sortBy}</span>

            <select

                value={sortBy}

```

```

                                onChange={ (e) =>
setSortBy(e.target.value as "views_desc" | "views_asc")}

                                className="bg-transparent
text-pink-400 font-bold outline-none cursor-pointer"

                                >

                                <option value="views_desc"
className="bg-neutral-900 text-white">{t.viewsDesc}</option>

                                <option value="views_asc"
className="bg-neutral-900 text-white">{t.viewsAsc}</option>

                                </select>

                                </div>

                                </div>

                                </div>

                                { /* 2. ALBUMS AND SONGS CATALOG GRID */ }

                                <div className="space-y-8">

                                    {filteredAlbums.map((album) => {

                                        const albumArtist = artists.find((a) => a.id
=== album.artistId) || artists[0];

                                        const albumSongs = songs.filter((s) =>
s.albumId === album.id);

                                        const sortedSongs = [...albumSongs].sort((a,
b) => {

                                            if (sortBy === "views_asc") return
a.totalViews - b.totalViews;

                                            return b.totalViews - a.totalViews;

                                        });

                                        if (sortedSongs.length === 0) return null;

                                        return (

```

```

<motion.div
  key={album.id}
  initial={{ opacity: 0, y: 12 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.4 }}
  className="rounded-3xl border
border-white/10 glass-panel p-6 shadow-xl space-y-5 relative
overflow-hidden"
>

  { /* ALBUM HEADER SECTION */ }

  <div className="flex flex-wrap
items-center justify-between gap-4 border-b border-white/10
pb-4">

    <div className="flex items-center
gap-4">

      <div className="relative w-14
h-14 rounded-2xl overflow-hidden border border-white/20 shadow-md
flex-shrink-0">

        <img src={album.coverUrl}
alt={album.titleAlbum} className="w-full h-full object-cover" />

      </div>

      <div>

        <div className="flex
items-center gap-2">

          <span
className="text-xs font-bold text-pink-400 uppercase
tracking-widest font-mono">

            {albumArtist.name}

          </span>

          <span
className="text-[10px] text-neutral-500 font-mono">&bull;
{album.year}</span>

        </div>

        <h2 className="text-lg
font-heading font-bold text-white tracking-tight">

          {album.titleAlbum}

```

```

        </h2>

    </div>

</div>

        <div className="flex items-center
gap-2 font-mono text-xs text-neutral-400">

            <Disc className="w-4 h-4
text-purple-400" />

            <span>{albumSongs.length}
{t.tracksInAlbum}</span>

        </div>

    </div>

    </div>

    { /* SONGS GRID FOR THIS ALBUM */ }

    <div className="grid grid-cols-1
sm:grid-cols-2 gap-3 sm:gap-4">

        {sortedSongs.map((song) => (

            <motion.div

                key={song.id}

                whileHover={{ scale:
1.01, y: -2 }}

                transition={{ duration:
0.2 }}

                onClick={() =>
onSelectSong(song.id) }

                className="group relative
rounded-2xl border border-white/10 bg-neutral-900/60
hover:bg-neutral-900/95 p-4 transition duration-300
cursor-pointer shadow-md hover:shadow-2xl overflow-hidden flex
flex-col justify-between"

                style={{

                    borderColor:
`${song.cardColor}40`,

                }}

            >

```

```

using card color */}

        className="absolute
inset-0 opacity-0 group-hover:opacity-15 transition duration-500
pointer-events-none"

        style={{
backgroundColor: song.cardColor }}

        />

        {/* TOP HALF: Song Info &
Artwork */}

        <div className="flex
items-start gap-3.5 relative z-10 mb-3">

            <div
className="relative w-16 h-16 rounded-xl overflow-hidden border
border-white/15 flex-shrink-0 group-hover:border-white/40
transition">

                <img
src={song.coverUrl} alt={song.titleSong} className="w-full h-full
object-cover" />

                {/* Play button
overlay */}

                <button

                    onClick={ (e)

=> {

e.stopPropagation();

                    if

(onPlaySong) onPlaySong(song);

                    }}

                >

            <div

className="w-8 h-8 rounded-full bg-white text-black flex
items-center justify-center shadow">

```

```

<Play
className="w-4 h-4 fill-black translate-x-0.5" />

</div>

</button>

</div>

<div
className="flex-1 min-w-0 space-y-1">

  <div
className="flex items-center justify-between gap-2">

    <h3
className="text-base font-subtitle font-bold text-white truncate
group-hover:text-pink-300 transition">

      {song.titleSong}

    </h3>

    <span
className="text-[10px] font-mono font-bold text-neutral-400
bg-black/40 px-2 py-0.5 rounded border border-white/10
flex-shrink-0">

      {song.likesFormatted} likes

    </span>

  </div>

  <div
className="flex items-center gap-2 text-xs font-mono
text-neutral-400">

    <span
className="text-neutral-300 font-bold">{(song.totalViews /
1000000).toFixed(0)}M vistas</span>

    <span>&bull;</span>

    <span
className="text-emerald-400 font-bold">${(song.kpis.earnings /
1000000).toFixed(1)}M</span>

  </div>

</div>

```

```

        </div>

        { /* BOTTOM: ACTION BUTTON
*/}

        <div className="pt-2
border-t border-white/5 flex items-center justify-between
relative z-10">

            <span
className="text-[10px] font-mono text-neutral-500">

                {albumArtist.name} &bull; {song.year}

            </span>

            <div className="flex
items-center gap-1 text-xs font-mono font-bold text-pink-400
group-hover:text-pink-300 transition">

                <span>{t.viewProfile}</span>

                <ChevronRight
className="w-3.5 h-3.5 group-hover:translate-x-1 transition
duration-300" />

            </div>

        </div>

    </motion.div>

    )}

</div>

</motion.div>

);

}}

</div>

</div>

```

```
    );  
  
}
```

- Explica tu proceso de pensamiento

Para la arquitectura de esta vista, el objetivo principal fue garantizar la **estabilidad de la aplicación** y el diseño

- Identifiqué desde el inicio que era indispensable contar con un selector o menú de canciones antes de renderizar la vista de detalle. Permitir el acceso sin una canción seleccionada habría causado un error.
- Analizando el esquema de los *mock data*, noté una relación de dependencia clara: el álbum depende del artista seleccionado, y las canciones dependen de esos álbumes y artistas. Para evitar "*estados imposibles*" (como mostrar un álbum asociado al artista equivocado), diseñe un filtrado que se limpia automáticamente al cambiar de opción.
- Implementé un filtrado por álbum complementado con ordenamiento basado en los KPIs de cada canción.

- **Reflexiona sobre la calidad de la ayuda brindada por la IA.**

La asistencia de la IA fue muy positiva. Logró respetar fielmente la arquitectura visual y tecnológica del proyecto (uso de **Tailwind CSS**, **Framer Motion** y acabados en **Glassmorphism**), integrando nuevas funciones sin romper la base preexistente.

Sin embargo, en el aspecto estético, las propuestas de la IA para la sección de KPIs no eran visualmente estéticas. Tras evaluar el impacto en la experiencia de usuario, tomé la decisión técnica de remover ese componente en favor de una interfaz más limpia, funcional y minimalista. A excepción de ese detalle, la IA fue una herramienta sumamente eficaz para estructurar una solución funcional y correcta.